



Which car should you get?

Dispatch rules for destination-controlled elevators, measured

elevator-dispatch-sim · one engineer, one AI assistant, one brief

The problem in plain language

Destination dispatch

You enter your destination before you board. The controller picks one car, at that moment, and the choice is final.

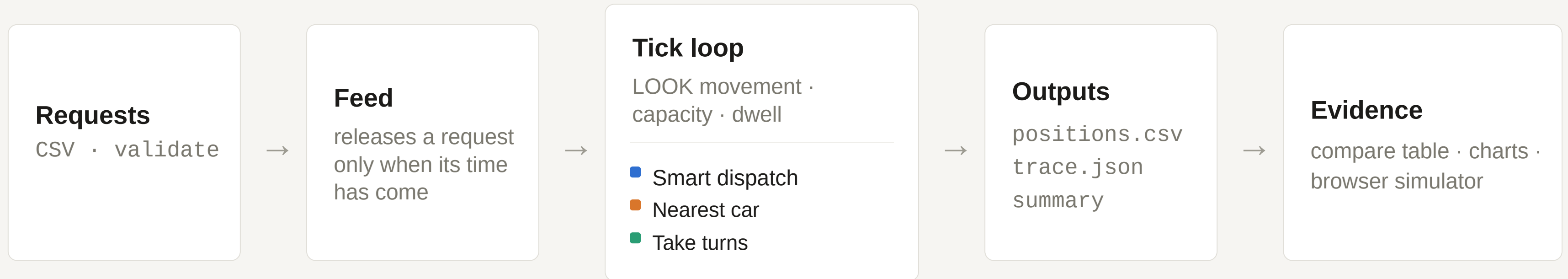
What good means

Short **wait** — request to boarding.
Short **total time** — request to arrival.
And **fairness** : no one waits far longer than everyone else.

Why it is not obvious

The controller cannot see the future. And the goals pull apart: the best average bundles people onto a busy car, which is efficient on the whole and hard on whoever keeps being passed over.

What was built



3

rules behind one interface

7

seeded traffic patterns · 20
fresh seeds each

192

tests, six levels

0

runtime dependencies

How the work was done

seven phases over two days

01	02	03	04	05	06	07
Frame	Plan	Research	Build & test	Measure	Validate	Document
The brief read closely, four questions answered.	Eleven ambiguities pinned; working rules; checkpoints.	How real controllers assign; traffic patterns from the literature.	Engine, three rules, seeded scenarios, 192 tests.	Comparison, charts, car usage, the browser simulator.	Two reviews by review passs of the same model; twenty fresh seeds per pattern.	Whitepaper, assumptions, design notes, this deck.
owner	owner + assistant	assistant	assistant, autonomous	assistant	review passs, same model	owner + assistant

OWNER	ASSISTANT
Intent, scope and quality bar. Every judgement call. Review at each checkpoint before anything was committed.	Drafted, researched, built, tested, challenged. Executed the plan autonomously, committing as it went.

Time: two sessions on consecutive days — about two hours of my attention on the first, four on the second; the assistant ran unsupervised for a further three to four hours across the phases.

How the model works

for the engineers • skippable

ONE TICK, IN FIXED ORDER

- 1 **Release and assign** — requests due now reach the scheduler; it returns a car index.
- 2 **Unload, then load** — at the current floor, in request order, while capacity allows, only in the departure direction.
- 3 **Log** — one row of positions per tick, from tick 0.
- 4 **Advance** — each car moves one floor toward its next stop, dwells, or stays idle.

ENFORCED BY STRUCTURE, PINNED BY TESTS

No peek-ahead. The feed releases a request only when $\text{time} \leq \text{now}$; a spy scheduler proves it.

Assignment is final, as in real destination systems — so capacity is the scheduler's problem.

LOOK movement. Keep direction while a stop lies ahead; never carry anyone away from their destination.

A stop costs one tick. The brief's literal model is recovered with `--dwell 0`.

The three dispatch rules

Smart dispatch

ETD · estimated time to destination

Picks the car that delays everyone the least: the newcomer's projected time to destination plus the delay added to passengers already assigned.

Trade-off: optimises the aggregate; without a fairness weight it can leave one person waiting a long time.

Nearest car

baseline · distance

Sends the closest car going the right way; a large penalty if the car is moving away or in the wrong direction.

Trade-off: ignores how many stops the car already has — so it degrades as load rises.

Take turns

baseline · round robin

Cars take requests in strict rotation, wherever they are. It uses no information at all.

Trade-off: nobody is singled out for a long wait; everybody waits longer.

Three policies sit on top of any rule: a **fairness weight** that makes long waits progressively more expensive to extend, **parking** idle cars at a chosen floor, and **zoning** cars to a range of floors.

How the numbers were assured

192

tests, six levels

- 1 Unit — every observable assumption
- 2 Structural — no peek-ahead
- 3 Property-based invariants, random buildings
- 4 Golden statistics — every committed result
- 5 Scenario and CLI, end to end
- 6 JS / Python conformance — incl. per-car usage

50_{/50}

scenario × rule combinations

The browser engine is a JavaScript port. Positions per tick, the full event list and every passenger's lifecycle are asserted identical to Python's.

41

findings, adversarial review

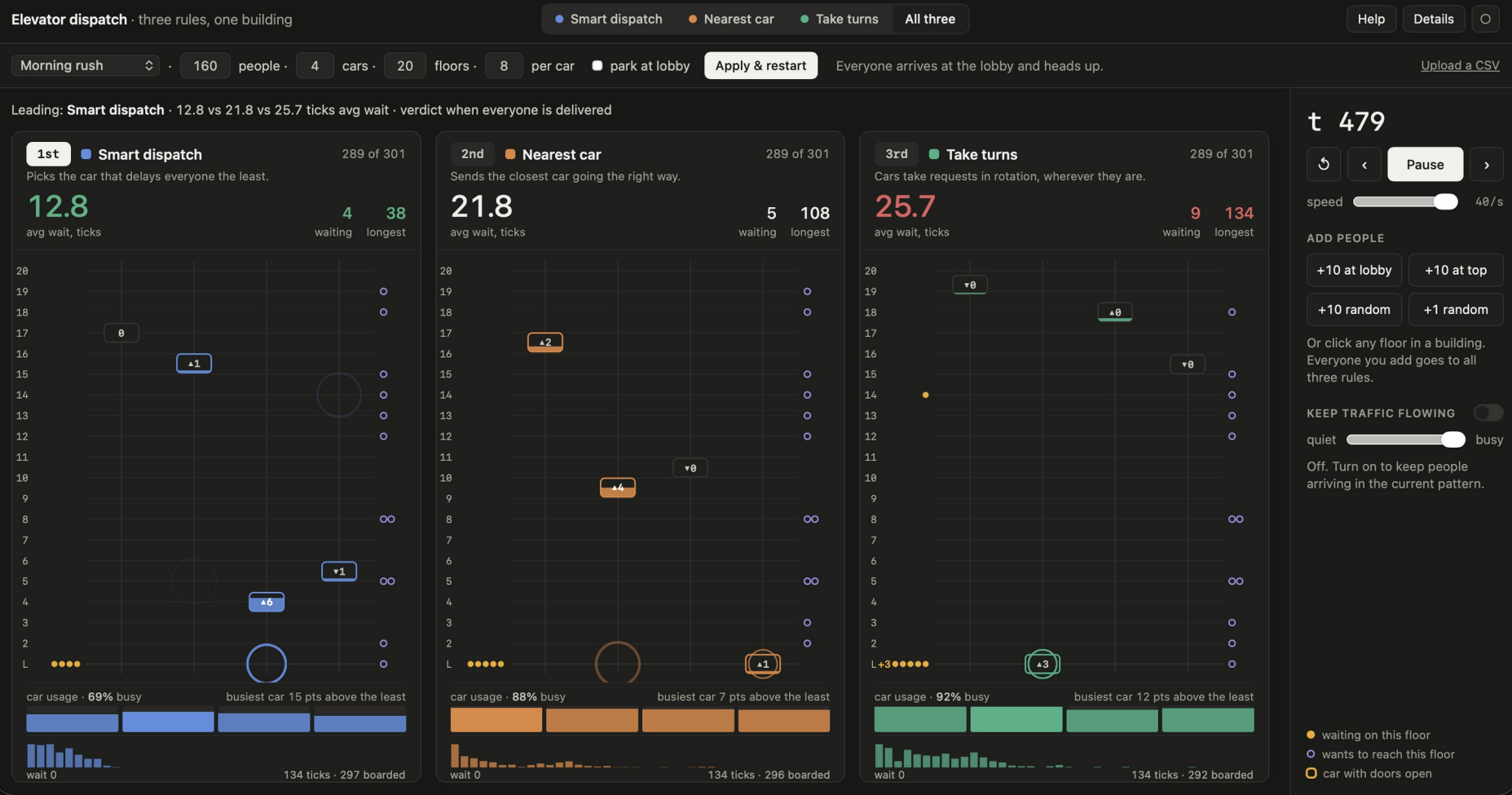
- 0** blockers · 8 major
- 16** changed code or tests
- 21** changed documents
- 4** resolved otherwise
- 20** fresh seeds per pattern — the headline held

The live simulator

`samuelgregorovic.github.io/elevator-dispatch-sim/docs/viewer/`

WHAT TO WATCH FOR

- Three rules, one building, the same 160 people — the **ranking** updates live.
- Amber dots** are people waiting. Watch where they pile up under each rule.
- The **usage strip** under each building: one bar per car, filled by busy share — see which cars sit out.
- Click a floor to **add a passenger**; tick **park at lobby** and restart.



THE QUESTION

How should a bank of elevators decide which car takes which passenger — and how much does the choice matter?

THE ANSWER, MEASURED

1.2–2.6x

shorter average wait with **smart dispatch** than **nearest car**, across twenty fresh seeds per pattern — 3.7x on the committed tower draw

2–3x

shorter than **take turns** — lowest average wait in 119 of 120 runs across the six realistic patterns

Average wait per rule, per pattern

ticks · lowest in bold

Scenario	Smart	Nearest	Turns
Morning up-peak · 20 floors, 4 cars	7.1	12.0	16.5
Lunch two-way · 20 floors, 4 cars	4.0	6.9	13.2
Evening down-peak · 20 floors, 4 cars	7.6	10.8	17.9
Quiet interfloor · 20 floors, 4 cars	3.9	4.5	8.1
Tower · 51 floors, 6 cars	18.3	67.1	51.4
Lobby burst · 20 floors, 2 cars × 6	60.3	76.2	60.1

TWO CASES THAT GO THE OTHER WAY

Lobby burst: within noise

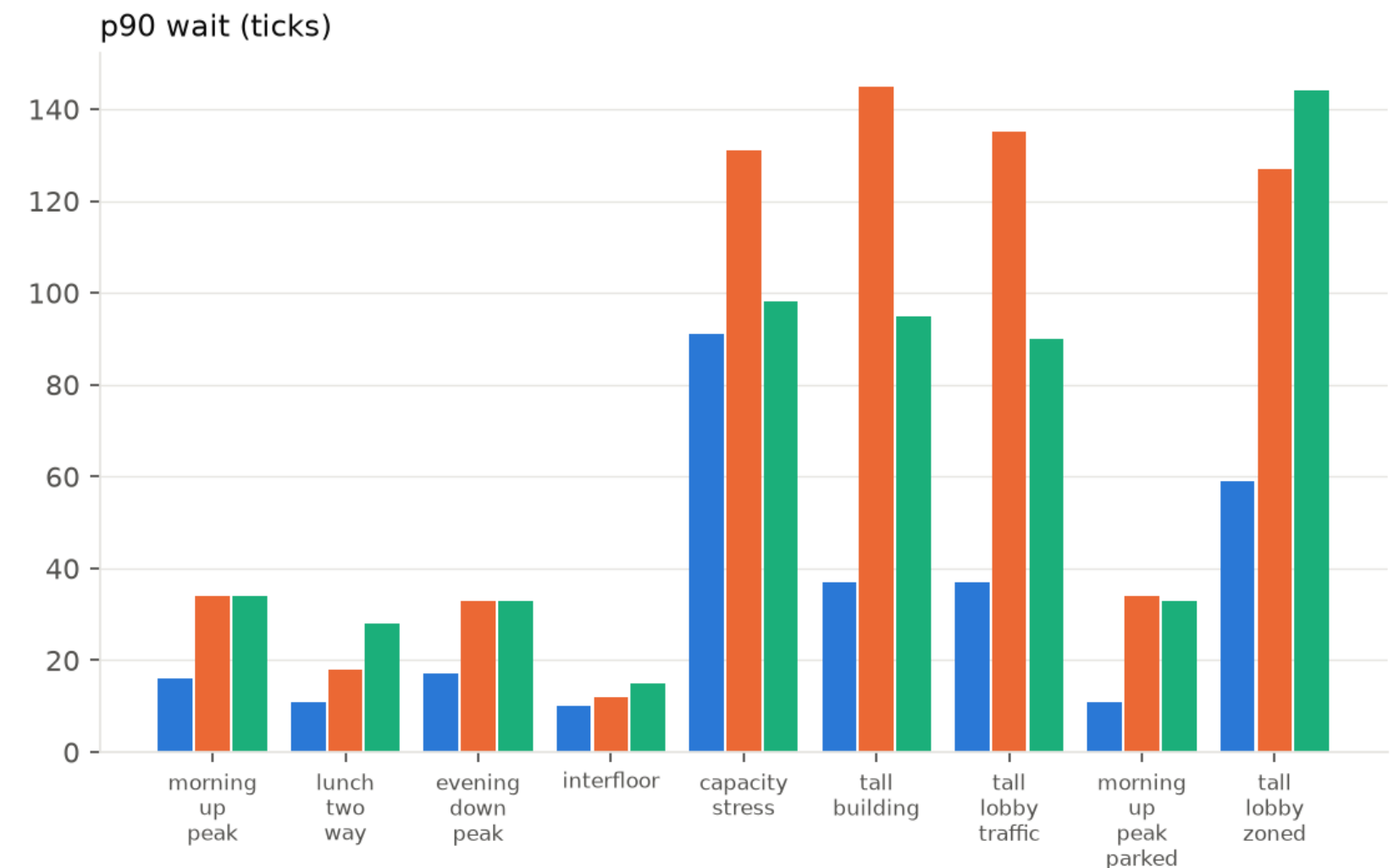
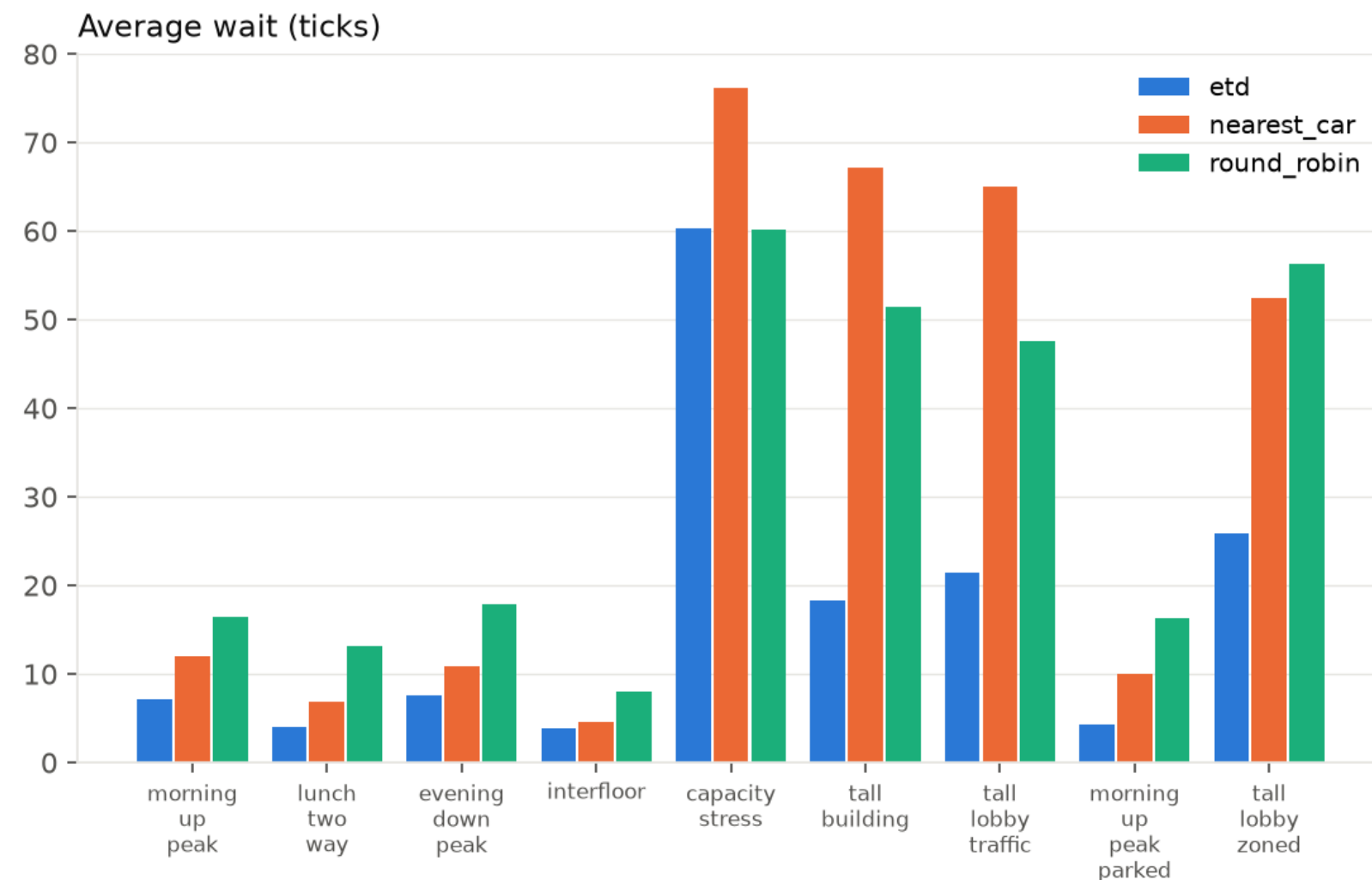
60.3 against 60.1 on the committed draw; across twenty seeds 55 ± 21 , 61 ± 20 and 59 ± 19 — within noise, smart dispatch lowest in 12 of 20. Every request has the same origin and the system is saturated; the rule does not matter here.

The brief’s sample: nearest car wins

6.3 against 15.0 on three passengers. Smart dispatch splits two lobby passengers across both cars to save a stop, leaving no idle car for the third request. A greedy rule with no view of the future can lose on tiny inputs — which is why the scenarios, not the sample, are the basis for judgement.

Waiting time by scenario

ticks · identical traffic per rule

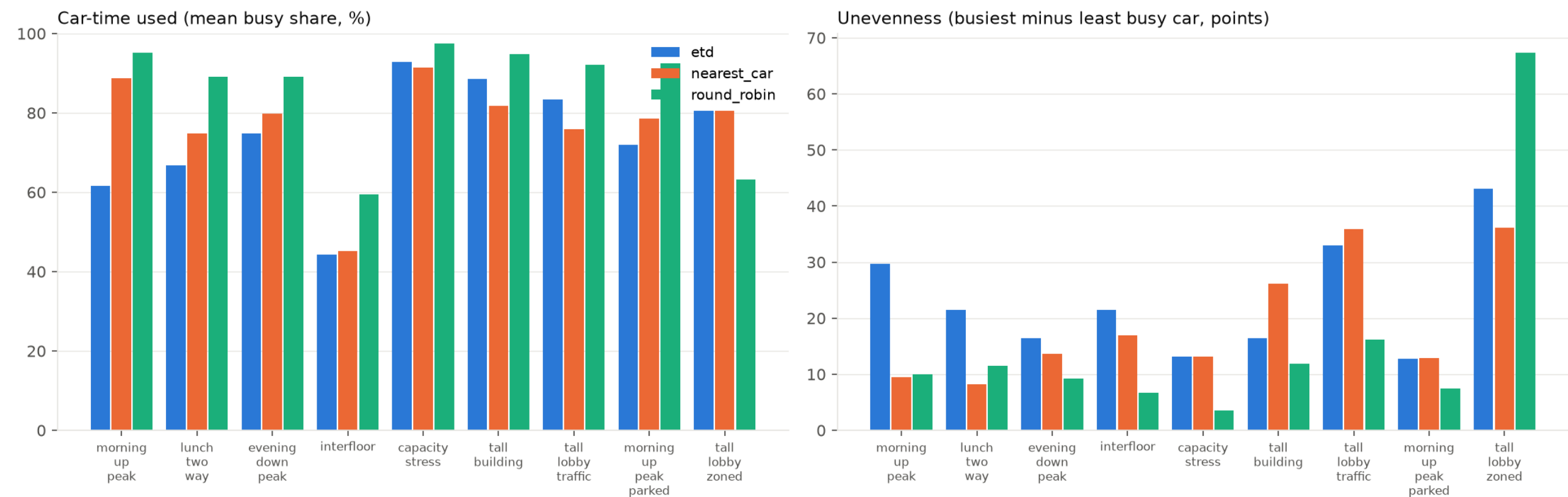


■ Smart dispatch ■ Nearest car ■ Take turns

Lowest on every realistic pattern; the margin grows with height and load.

How the cars are used

busy = aboard or assigned · spread = busiest – least busy car



62% vs 89% vs 95%

Least car-time for the same job. Cars busy in the morning peak. It bundles: 6.1 floors of car travel per passenger against 9.3 and 10.0 — and its riders sit through more stops, not fewer (1.76 a trip vs 1.21 and 0.95).

36% fair share 25%

But the work is shared least evenly. One car in four carries 36% of the morning passengers.

30 → 18 · 21 → 5 · 33 → 10

The fairness weight evens the cars as a side effect. Spread in points, committed draws; narrower in 11–15 of 20 fresh seeds. One knob, two effects.

Which rule, which policy, for which building average wait, ticks

Building and traffic	Choose	Evidence
Ordinary office, moderate traffic · ≤20 floors	Smart dispatch. 1.5–2× shorter average wait than nearest car on the three peaks across seeds; 29–42% on the committed draws.	Strong · 20 of 20 seeds on each peak
Quiet, interfloor traffic	Smart dispatch , but by 1.2× — a nearest-car controller is within 15% here.	19 of 20 seeds
Tall building · ~50 floors	Smart dispatch — 2.6× across seeds (3.7× committed draw); nearest car’s maximum 250 ± 85 against 65 ± 13.	Strong · 20 of 20 seeds
Where the tail matters · hospitals, hotels	The fairness weight works reliably only under saturation: burst max lower in 14 of 20. The tower’s 108 → 56 was one draw — max lower in 4 of 20, average up in 18.	Robust on the burst · not reproduced on the tower
Morning up-peak, known lobby	Park idle cars at the lobby. 7.1 → 4.3 average on the committed draw; −38.6 ± 11.6% across seeds. Should follow time of day.	20 of 20 seeds
Even wear across cars · per-car maintenance	Add the fairness weight — it narrows the spread between cars. Do not switch to take-turns: it evens cars by working all of them more.	Majority of seeds · 11–15 of 20 · wear not modelled
Saturation · more people than cars can move	No rule fixes a capacity problem — the three rules are within noise. The levers are cars and capacity.	20 seeds · same answer: the rule does not matter
Zoning and express cars	A null result at moderate load: 24.4 vs 24.8 across seeds, worse in 11 of 20. The committed 21.4 → 25.8 was one draw.	Null result, 20 seeds · positive case untested

Limits of the model

NOT MODELLED

Door and acceleration physics — a stop is one tick, whoever boards
Reassignment after dispatch, as hybrid systems allow
Sky-lobby transfers across zones
Passenger no-shows and group entries
Energy

WHAT WOULD SHARPEN THE CONCLUSIONS

A sweep of arrival rate — where each rule breaks down
A sweep of car count — the price of capacity
Fairness weight swept on more than two scenarios
The saturated counterpart to the zoning experiment
Batch re-optimisation in place of greedy per-request choice

The headline survived twenty fresh re-draws of every pattern. Measuring seed variance changed two secondary claims — the fairness weight on the tower, and zoning — which is what the remaining sweeps would do to the rest.

What I would do next

BUILD, IN THIS ORDER

- 1 **Reassignment window** — move a passenger until their car starts slowing, as hybrid systems do.
- 2 **Batch re-optimisation** — revisit outstanding assignments every few ticks.
- 3 **Time-of-day policies** — parking floor and fairness weight follow the traffic profile.

QUESTIONS FOR A REAL CUSTOMER FIRST

What does your traffic look like — arrival rate and mix, by hour?

Which complaint are we fixing: the average wait, or the worst one?

Is a zoned or sky-lobby bank on the table, or one free group?

May the display change a passenger's car after it is shown?

Do people travel in groups, and how often do they not show up?

A rule that accounts for queued stops and the delay it inflicts on others beats rules that do not — increasingly so as buildings get taller and busier.

REPOSITORY

github.com/samuelgregorovic/elevator-dispatch-sim

LIVE SIMULATOR

samuelgregorovic.github.io/elevator-dispatch-sim/docs/viewer/